

## TECNOLOGÍAS DE LA INFORMACIÓN

### Desarrollo de Sistemas Informáticos

4 créditos



#### Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

| Titulaciones                                    | Semestre      |
|-------------------------------------------------|---------------|
| • <i>Tecnologías de la Información en Línea</i> | <i>Quinto</i> |

**NOMBRES Y APELLIDOS:** Marcel Ismael Sinche Yumisaca

**PARALELO:** "A"

**SEMESTRE:** Quito semestre

**ACTIVIDAD:** Actividad 1

PERIODO FEBRERO 2026 - AGOSOT 2026



## SISTEMA DE GESTIÓN DE INCIDENTES TÉCNICOS DISEÑO TEÓRICO COMPLETO

### 1. Introducción

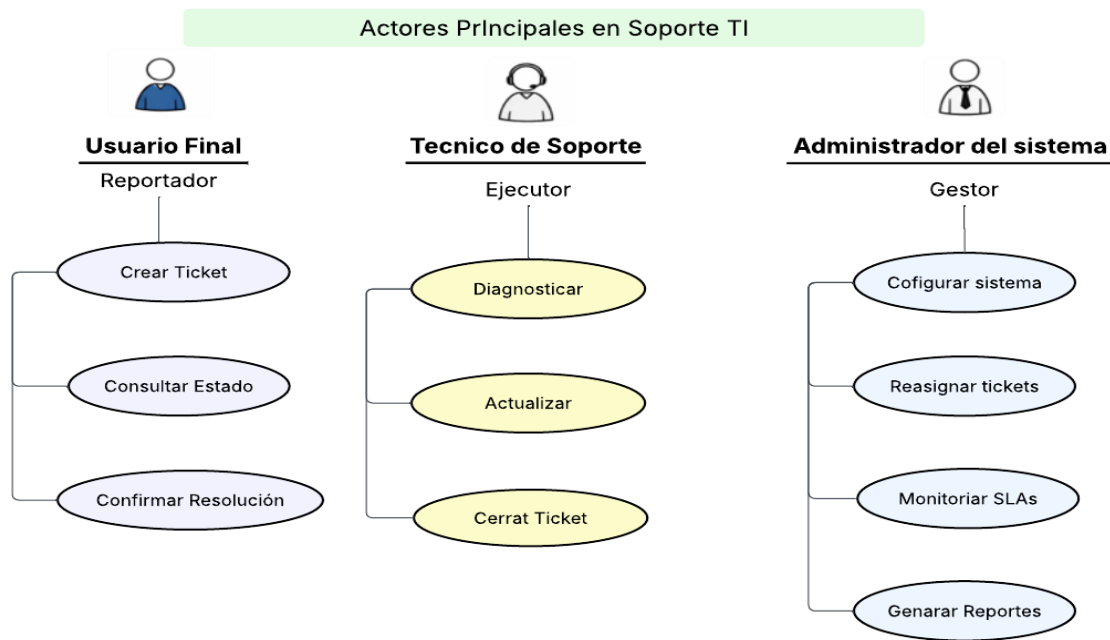
El contexto expone el diseño teórico de un sistema Help Desk (balcón de ayuda o apoyo informático) para la gestión de incidentes y demandas en un Centro de Datos. La solución tiene por objetivo general la mejora de los procesos de administración del registro, categorización, monitoreo, escalación y resolución de fallas relacionadas con la infraestructura tecnológica de modo que se asegure en la administración de servicios de soporte técnico eficiente, centralizada, documentada y trazable.

La arquitectura del sistemas está basada en principios estable de arquitectura de software, que proporciona una estructura y modularización que permite escalabilidad, mantenibilidad y reutilización de componentes . También se aprovecha un modelo de control de versiones con la metodología Gitflow, para el manejo organizado del ciclo de desarrollo, integración, entrega y despliegue del software. Igualmente se adoptan patrones de diseño y buenas prácticas de desarrollo para el incremento en la robustez, disponibilidad y evolución ante futuras necesidades operativas del sistema en el entorno tecnológico.

### 2. Análisis del Problema

**2.1 Se identifica los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas de cuerdo al siguiente detalle:**

| Actor                     | Rol        | Responsabilidades                                                    |
|---------------------------|------------|----------------------------------------------------------------------|
| Usuario Final             | Reportador | Crea tickets, consulta estado, confirma resolución                   |
| Técnico de Soporte        | Ejecutor   | Recibe asignación, diagnostica, actualiza estado, cierra ticket      |
| Administrador de Sistemas | Gestor     | Configura sistema, reasigna tickets, monitorea SLAs, genera reportes |



### 2.1.1 Usuario Final

El usuario es aquel que consume los servicios que ofrece un Data Center y quien debe notificar cuando se presentan incidentes en dichos servicios ante cualquier anomalía o falla de la infraestructura tecnológica, sea en hardware, conectividad de red, sistemas operativos o aplicaciones de software. Su rol es esencial para comenzar el proceso de atención, diagnóstico y solución de incidentes dentro del sistema Help Desk

#### Funciones principales

- Crear tickets de soporte.
- Describir el problema presentado.
- Consultar el estado del incidente.
- Recibir notificaciones sobre avances o soluciones.
- Confirmar el cierre del ticket.

Facilitar el registro ágil, estructurado y centralizado de incidentes tecnológicos, permitiendo una gestión eficiente de fallos y optimizando los tiempos de atención, seguimiento y resolución por parte del equipo de soporte técnico.



### **2.1.2 Técnico de Soporte**

El profesional encargado de administrar, monitorear, diagnosticar y resolver los incidentes del sistema en la mesa de ayuda cuyo fin es asegurar que el servicio tecnológico se mantenga operativo se llama Técnico en Soporte, y esto lo logra aplicando procedimientos técnicos, herramientas y protocolos de asistencia especializados.

#### **Funciones principales**

- Revisar tickets asignados.
- Diagnosticar problemas técnicos.
- Actualizar el estado del incidente.
- Registrar soluciones aplicadas.
- Escalar incidentes complejos.
- Cerrar tickets resueltos.

Garantizar la resolución eficiente y oportuna de incidentes técnicos, asegurando la continuidad operativa, disponibilidad y estabilidad de los servicios e infraestructura tecnológica del Data Center.

### **2.1.3 Administrador de Sistemas**

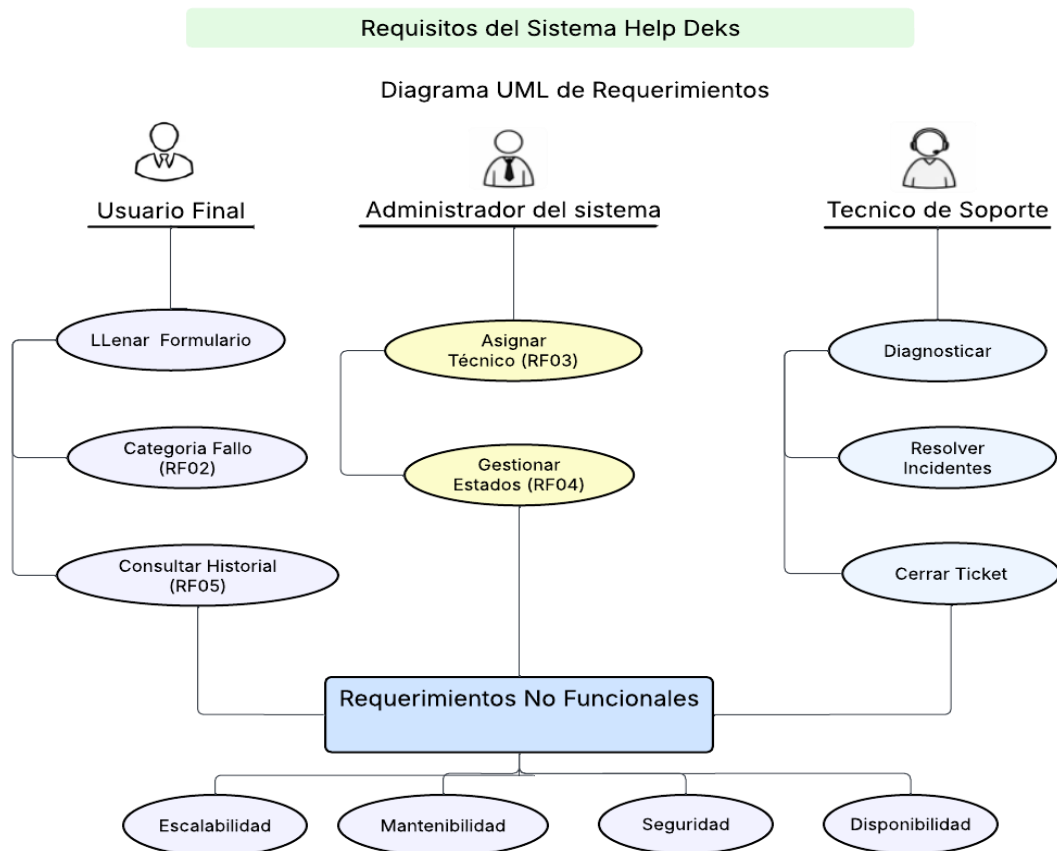
Está encargado de gestionar, configurar y administrar el sistema Help Desk en todos sus aspectos, garantizará la correcta administración de usuarios, incidencias, recursos y procesos operativos en el entorno del Data Center.

#### **Funciones principales**

- Gestionar usuarios y permisos.
- Asignar técnicos especializados.
- Configurar categorías y prioridades.
- Supervisar el desempeño del soporte técnico.
- Generar reportes y estadísticas.
- Controlar la seguridad y trazabilidad del sistema.

Garantizar el correcto funcionamiento, control y administración del sistema de soporte técnico, asegurando una gestión organizada, eficiente y segura de los procesos de atención, seguimiento y resolución de incidencias.

## 2.2 Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).



### 2.2.1 Requerimientos Funcionales

Los requerimientos de funcionamiento describen las funciones, procesos y servicios que el sistema debe proporcionar para cumplir con las necesidades operativas del usuario y para el mantenimiento de la plataforma Help Desk.

| ID    | Requerimiento                                                             | Actor                 |
|-------|---------------------------------------------------------------------------|-----------------------|
| RF-01 | Crear y registrar ticket de incidente                                     | Usuario Final         |
| RF-02 | Categorizar fallo (Hardware / Red / Software)                             | Usuario Final / Admin |
| RF-03 | Asignar técnico especializado por categoría                               | Admin / Sistema       |
| RF-04 | Gestionar estados del ticket (Abierto → En Progreso → Resuelto → Cerrado) | Técnico / Admin       |
| RF-05 | Notificar automáticamente al técnico asignado                             | Sistema               |
| RF-06 | Escalar incidentes no resueltos según SLA                                 | Sistema / Admin       |



|       |                                                  |       |
|-------|--------------------------------------------------|-------|
| RF-07 | Consultar historial y trazabilidad de incidentes | Todos |
| RF-08 | Generar reportes de desempeño                    | Admin |

#### 2.2.1.1 RF01 – Crear Ticket

El sistema debe permitir al usuario final registrar incidentes mediante un formulario estructurado, para que pueda proporcionar información relevante sobre el incidente, como una descripción del problema, categoría, prioridad, fecha y recursos afectados.

##### **El ticket debe incluir:**

- Título del problema.
- Descripción detallada.
- Tipo de fallo:
  - Hardware
  - Red
  - Software
- Nivel de prioridad.
- Fecha y hora del reporte.

Permitir el registro, almacenamiento y documentación adecuada de los incidentes presentados en el Data Center, garantizando la disponibilidad de información precisa para su seguimiento, análisis y resolución oportuna.

#### 2.2.1.2 RF02 – Categorizar Fallo

El sistema tendrá que clasificar automáticamente los incidentes al nivel de categoría, prioridad y tipo de error para permitir un mejor tratamiento, seguimiento y asignación a los técnicos.

##### **Categorías principales**

- Fallos de hardware.
- Fallos de red.
- Fallos de software.

Organizar y categorizar los tickets de soporte de manera estructurada, permitiendo un mejor control operativo, una atención



especializada y una gestión más eficiente de los incidentes reportados.

#### **2.2.1.3 RF03 – Asignar Técnico**

El administrador del sistema puede asignar incidentes al personal de técnica especializada, para cada categoría, en función de prioridad y complejidad del problema, con el fin de aprovechar mejor los tiempos de respuesta.

##### **El sistema deberá:**

- Mostrar técnicos disponibles.
- Relacionar especialidades con categorías.
- Registrar fecha y hora de asignación.

Gestión y resolución de incidencias por personal técnico cualificado y especializado en la materia, con la garantía de calidad en la atención y en los resultados, adaptados al tipo de problema o incidencia reportada.

#### **2.2.1.4 RF04 – Gestionar Estados del Ticket**

El sistema debe permitir que el estado de los incidentes se actualice dinámicamente durante todo su ciclo de vida, con lo que será posible seguir el proceso de atención y resolución de manera continua.

##### **Estados posibles**

- Abierto
- En proceso
- Escalado
- Resuelto
- Cerrado



Dar seguimiento permanente y actualizado al estado y avance de cada ticket, permitiendo el control eficiente de las actividades que se realizan en el proceso de atención y solución del incidente.

#### **2.2.1.5 RF05 – Consultar Historial**

El sistema deberá posibilitar que tanto usuarios como administradores puedan acceder a todo el historial de modificaciones de los tickets, desde los cambios de estado, asignaciones, actualizaciones y acciones realizadas durante la vida del incidente.

Los usuarios y administradores podrán consultar el historial de cambios realizados en los tickets.

##### **Información registrada**

- Cambios de estado.
- Técnico asignado.
- Comentarios.
- Fecha y hora de modificaciones.

Garantizar la trazabilidad, monitoreo y control integral de las incidencias, permitiendo el registro detallado de todas las acciones y cambios realizados durante el proceso de gestión y resolución de los tickets.

#### **2.2.2 Requerimientos No Funcionales**

Los requerimientos no funcionales especifican los atributos de calidad, las restricciones técnicas y los criterios de desempeño necesarios a fin de que el sistema sea eficiente, confiable, seguro y se ejecute de manera apropiada.

| ID     | Requerimiento             | Métrica                                                              |
|--------|---------------------------|----------------------------------------------------------------------|
| RNF-01 | Escalabilidad             | Soportar NNN tickets concurrentes sin degradación                    |
| RNF-02 | Mantenibilidad            | Módulos desacoplados, cambios en <1< 1<1 componente no afectan otros |
| RNF-03 | Trazabilidad de versiones | Gitflow completo con historial auditable                             |
| RNF-04 | Disponibilidad            | $\geq 99.5\% \wedge \geq 99.5\%$ uptime                              |
| RNF-05 | Tiempo de respuesta       | $< 500\text{ms}$ en operaciones de creación/consulta                 |
| RNF-06 | Seguridad                 | RBAC, autenticación por roles                                        |

#### 2.2.2.1 RNF01 – Escalabilidad

El sistema deberá contar con una arquitectura escalable que permita soportar el crecimiento futuro en número de usuarios, volumen de incidentes, recursos tecnológicos y funcionalidades, sin afectar el rendimiento ni la estabilidad de la plataforma. sistema debe soportar el crecimiento futuro en:

- Número de usuarios.
- Cantidad de tickets.
- Nuevos módulos o funcionalidades.

El asegurar un rendimiento, estabilidad y respuesta ópticos para el sistema al aumentar la cantidad de usuarios, eventos y servicios tecnológicos.

#### 2.2.2.2 RNF02 – Mantenibilidad

El sistema será diseñado para tener una arquitectura coherente, modular y desacoplada que permita el mantenimiento, escalabilidad, la introducción de nuevos componentes y permita la reutilización de funcionalidades del sistema de forma eficiente.

- Corrección de errores.
- Actualización de funcionalidades.
- Reutilización de código.



### **Medidas aplicadas**

- Uso de patrones de diseño.
- Separación por capas.
- Documentación técnica.

Reducir costos y tiempo de mantenimiento.

### **2.2.2.3 RNF03 – Trazabilidad de Versiones**

El proceso de desarrollo deberá implementar la metodología Gitflow dentro de un repositorio en GitHub, permitiendo una gestión estructurada de ramas, control de versiones, integración continua y administración eficiente del ciclo de vida del software..

### **Ramas principales**

- main
- develop
- feature/\*
- release/\*
- hotfix/\*

Con el fin de asegurar un control eficiente de las modificaciones, trazabilidad del historial de versiones y una gestión del trabajo colaborativo bien organizada, que permita la integración, seguimiento y mantenimiento del desarrollo del sistema.

### **2.2.2.4 RNF04 – Seguridad**

El sistema deberá proveer autenticación y control de acceso basado en roles de usuario, garantizando que cada usuario sólo tenga acceso a las funciones y recursos a los que está autorizado con base en sus permisos y responsabilidades.

Preservar la integridad, confidencialidad y disponibilidad de la información del sistema, impidiendo accesos no autorizados y aumentando la seguridad de los datos almacenados en la plataforma de Help Desk.



#### **2.2.2.5 RNF05 – Disponibilidad**

El sistema debe estar diseñado para proporcionar un alto grado de disponibilidad y operación continua, dada la criticidad operativa del Data Center y la necesidad de mantenerse los servicios de soporte técnico activos en todo momento.

Garantizarse un acceso continuo y fiable a la plataforma de soporte que posibilite la atención continua de incidentes y solicitudes.

#### **2.2.2.6 RNF06 – Rendimiento**

El sistema debe realizar sus operaciones con tiempos de respuesta cortos y un buen rendimiento para proporcionar una interacción fluida al registrar, consultar, actualizar y hacer seguimiento de incidentes.

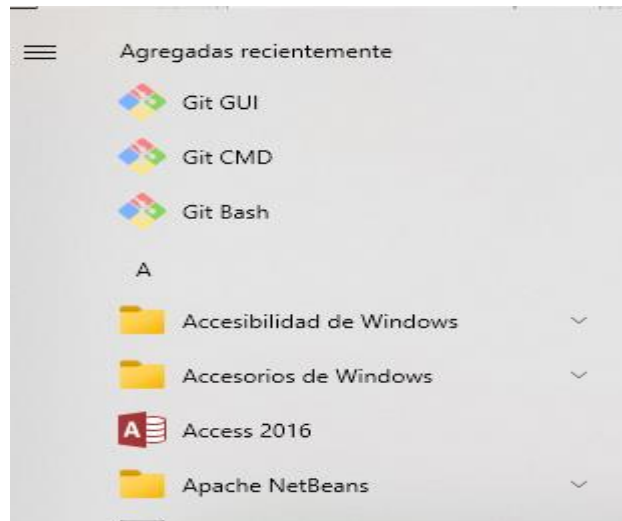
Reducir el retraso en los procedimientos de administración de tickets para garantizar la atención rápida y el seguimiento constante de los incidentes reportados.

### **3. Diseño del sistema**

**3.1** Se realiza la creación de un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).

#### **3.1.1 Instalación de Git en Windows**

- El menú de inicio mostrará la añadidura exitosa de las herramientas Git Bash, Git CMD, Git GUI, para gestionar y ejecutar operaciones de control de versiones.
- Con la sola que dispone para proveerla a las herramientas que gestionan los repositorios y versiones y trabajo colaborativo de desarrollo, asegurar su necesaria disponibilidad dentro del entorno del sistema.



### 3.1.2 Configuración de usuario en Git

- Introduce en la terminal estos son los comandos de configuración global para que se configuren los campos de usuario y correo electrónico, con esto se puede saber que en el historial del repositorio se le muestran a los autores de los commits, o sea que con cada commit que hagan en algún repositorio podrán saber de dónde o quién fue el responsable y podrán tener trazabilidad con que cambios hicieron y cuándo.

```
MINGW64:/c/Users/User
User@DESKTOP-AT91HB2 MINGW64 ~
$ git config --global user.name "MARCEL"
User@DESKTOP-AT91HB2 MINGW64 ~
$ git config --global user.email "msinche7445@utm.edu.ec"
User@DESKTOP-AT91HB2 MINGW64 ~
$ |
```

### 3.1.3 Inicialización del repositorio local

- Se ejecuta en la carpeta PROYECTO 2026 el comando `git init` para inicializar el repositorio local. A continuación, se añade el archivo del proyecto con `git add` y se efectúa el primer commit con `git commit`, con lo que se tiene una

primera versión de los documentos y comienza el control de versiones del proyecto desde el principio.

```
MINGW64:/c:/Users/User/Desktop/PROYECTO 2026

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026
$ git init
Initialized empty Git repository in C:/Users/User/Desktop/PROYECTO 2026/.git/

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026 (master)
$ git add .

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026 (master)
$ git commit -m "commit inicial: subida de la primera version de la presentacion"
[master (root-commit) a1cd66f] commit inicial: subida de la primera version de la presentacion
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Proyecto 1.pdf
create mode 100644 "SISTEMA DE GESTI\303\223N DE INCIDENTES T\303\211CNICOS.docx"

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026 (master)
$
```

### 3.1.4 Sincronización con GitHub

- Hay un proceso de merge para combinar los cambios que se han desarrollado en las distintas ramas del repositorio y después se hace un git push hacia el repositorio remoto en GitHub, con lo que se suben las actualizaciones locales, se comparte el trabajo con el equipo, y se sincronizan el repositorio local y remoto.

```
MINGW64:/c:/Users/User/Desktop/PROYECTO 2026

softwareMarcel-.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

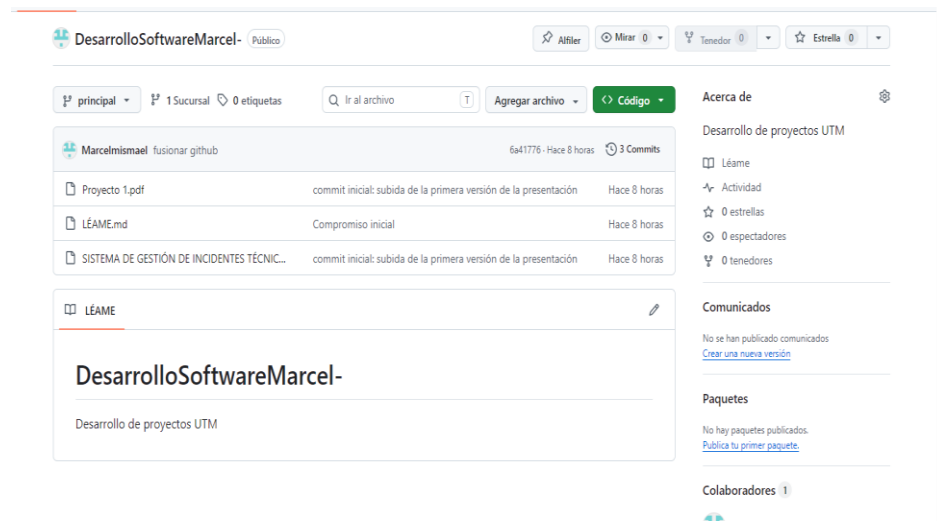
User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026 (main|MERGING)
$ git commit -m "merge github"
[main 6a41776] merge github

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026 (main)
$ git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 2.32 MiB | 1.25 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/marcelmismael/DesarrolloSoftwareMarcel-.git
 8846260..6a41776 main -> main
branch 'main' set up to track 'origin/main'.

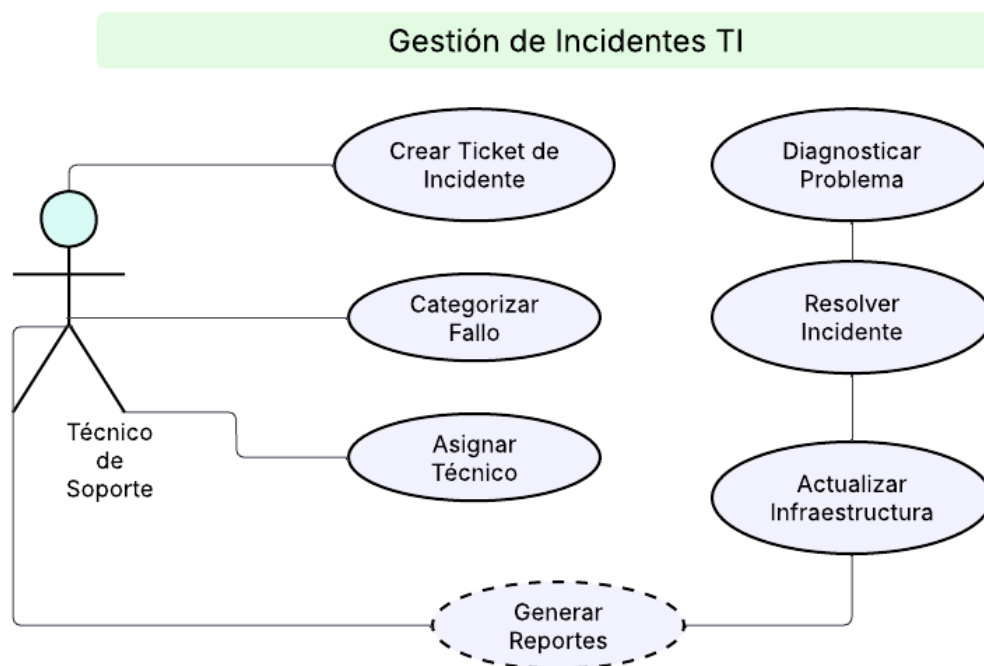
User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO 2026 (main)
```

### 3.1.5 Repositorio público en GitHub

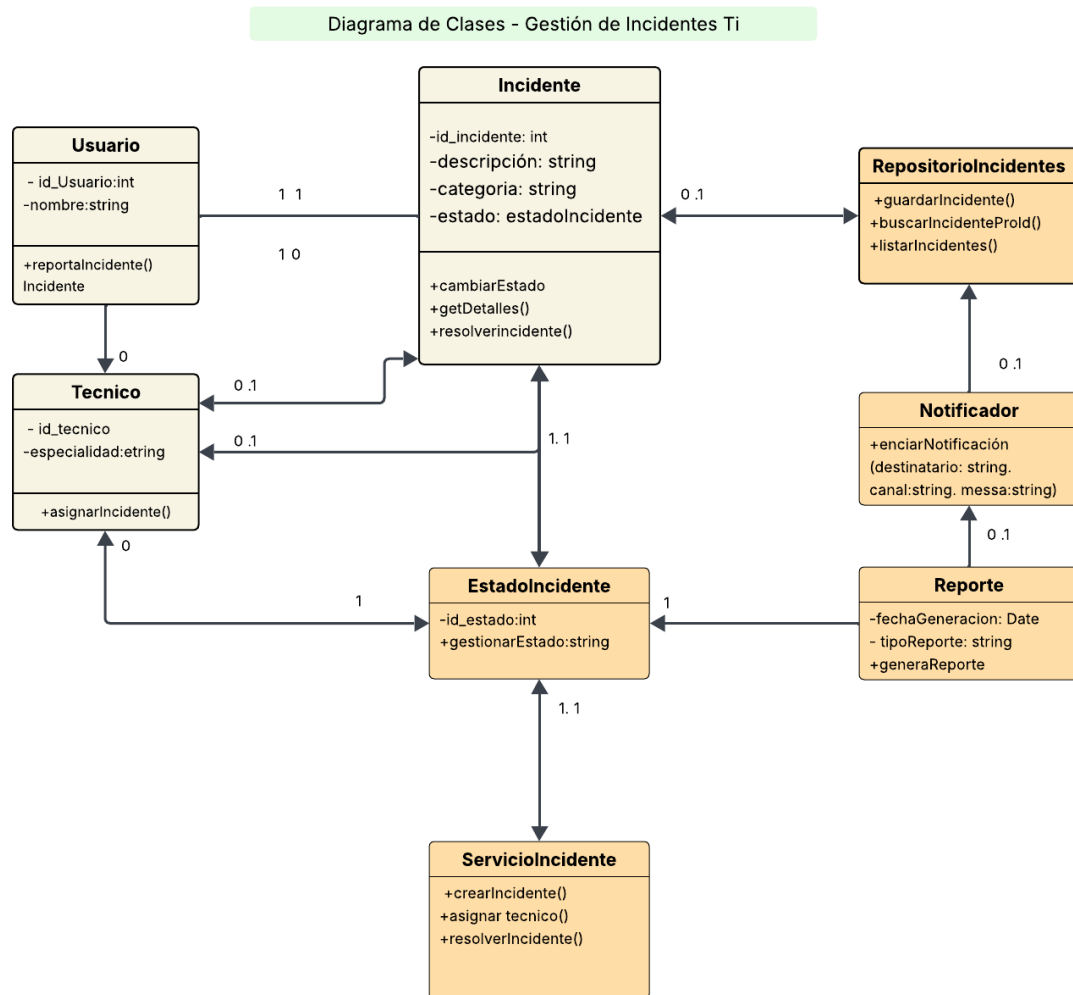
- Se observa el repositorio “DesarrolloSoftwareMarcel-” en GitHub con sus archivos, carpetas y commits registrados, y es posible centralizar el proyecto en la nube, facilitar el acceso público a la información y promover la colaboración y control de versiones entre los miembros del equipo.



### 3.2 Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.



### 3.3 Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.



#### 3.3.1 Instrucciones del gráfico técnica y descriptiva

El gráfico muestra la arquitectura orientada a objetos del sistema de gestión de incidentes para soporte, donde cada clase representa una entidad funcional concreta del proceso de registro, seguimiento, atención y resolución de incidentes. La organización representa la modularidad, encapsulación y separación de responsabilidades, permitiendo un sistema mantenible y escalable.

##### 3.3.1.1 Usuario

Representa al actor principal que registra fallos, solicitudes o incidencias dentro del sistema.



**Atributos:**

- id\_usuario
- nombre

**Método principal:**

- reportarIncidente(): permite crear y registrar una nueva instancia de la clase Incidente.

**Relación:**

- Un Usuario puede generar múltiples incidentes (1..\*), estableciendo una relación de asociación entre usuario e incidente.

**Función dentro del sistema:**

Actúa como punto inicial del flujo de soporte técnico, proporcionando la información necesaria para la creación y seguimiento de tickets.

### 3.3.1.2 Técnico

Encargado de diagnosticar, atender y resolver los incidentes reportados.

**Atributos:**

- id\_tecnico
- especialidad

**Método principal:**

- asignarIncidente(): asocia un incidente específico al técnico responsable de su atención.

**Relación:**

- Existe una relación de colaboración entre Técnico e Incidente, donde un técnico puede atender varios incidentes y un incidente



puede involucrar diferentes técnicos según el proceso de soporte.

**Función dentro del sistema:**

Garantiza la ejecución de tareas de soporte, seguimiento y solución de problemas técnicos.

**3.3.1.3 Incidente**

Constituye el núcleo funcional del sistema, representando cada evento técnico reportado.

**Atributos:**

- `id_incidente`
- `descripcion`
- `categoria`
- `estado`

**Métodos principales:**

- `cambiarEstado()`
- `getDetalles()`
- `resolverIncidente()`

**Relaciones:**

- Está asociado a un Usuario, quien genera el reporte.
- Se vincula con un Técnico, encargado de su resolución.
- Depende de la clase `EstadoIncidente` para gestionar su ciclo de vida.

**Función dentro del sistema:**

Centraliza toda la información relacionada con el proceso de soporte técnico, incluyendo seguimiento, asignación y resolución.



#### 3.3.1.4 EstadoIncidente

Administra las diferentes etapas del ciclo de vida de un incidente.

##### Estados posibles:

- Abierto
- En Progreso
- Resuelto
- Cerrado

##### Atributos:

- `id_estado`
- `nombreEstado`

##### Método principal:

- `gestionarEstado()`: Controla las transiciones y validaciones entre estados.

##### Relación:

- Un mismo estado puede estar asociado a múltiples incidentes.

##### Función dentro del sistema:

Permite mantener trazabilidad y control sobre el avance de cada ticket.

#### 3.3.1.5 ServicioIncidente

Representa la capa de lógica de negocio encargada de coordinar las operaciones principales del sistema.

##### Métodos piales:

- `crearIncidente()`
- `asignarTecnico()`
- `resolverIncidente()`



**Relaciones:**

- Interactúa directamente con `RepositorioIncidentes` para la persistencia de datos.
- Utiliza `Notificador` para enviar alertas y actualizaciones.

**Función dentro del sistema:**

Orquesta los procesos internos y actúa como intermediario entre las interfaces del sistema y las entidades del modelo.

### 3.3.1.6 RepositorioIncidentes

Componente encargado de la persistencia y administración de datos relacionados con los incidentes.

**Métodos principales:**

- `guardarIncidente()`
- `buscarIncidentePorId()`
- `listarIncidentes()`

**Relación:**

- Gestiona el almacenamiento y recuperación de múltiples incidentes.

**Función dentro del sistema:**

Aísla la lógica de acceso a datos, facilitando mantenimiento y compatibilidad con diferentes motores de base de datos.

**Notificador**

Módulo responsable de la comunicación y envío de alertas automáticas.

**Método principal:**

- `enviarNotificacion(destinatario, canal, mensaje)`



**Relación:**

- Un incidente puede generar múltiples notificaciones dirigidas a técnicos, administradores o usuarios.

**Función dentro del sistema:**

Mantiene informados a los actores involucrados sobre cambios, asignaciones o resoluciones de incidentes en tiempo real.

### 3.3.1.7 Reporte

Componente orientado al análisis, monitoreo y auditoría del sistema.

**Atributos:**

- `fechaGeneracion`
- `tipoReporte`

**Método principal:**

- `generarReporte()`

**Relación:**

- Puede generar informes basados en estados, técnicos, categorías o métricas de desempeño.

**Función dentro del sistema:**

Facilita la toma de decisiones mediante estadísticas, indicadores y reportes operativos del servicio de soporte técnico.

**3.4** Diseñar la arquitectura general del sistema integrando el flujo de versiones. Diseñar la arquitectura general del sistema integrando el flujo de versiones.

### Arquitectura general del sistema

#### Capa de presentación (Front-End)

- Interfaz web o aplicación donde los usuarios y técnicos interactúan.
- Tecnologías sugeridas: React, Angular o Vue.js.



- Funciones principales:
  - Registro de incidentes
  - Visualización de tickets
  - Panel de técnico

### Capa de lógica de negocio (Back-End)

- Implementa los casos de uso definidos en el diagrama UML.
- Lenguajes sugeridos: Java (Spring Boot) o Python (FastAPI).
- Módulos principales:
  - Gestión de incidentes
  - Asignación de técnicos
  - Notificaciones y reportes

### Capa de datos

- Base de datos relacional o híbrida según el volumen de información.
- Ejemplo: PostgreSQL para datos estructurados y MongoDB para logs o auditorías.
- Entidades principales: Usuario, Tecnico, Incidente, Estado, Reporte.

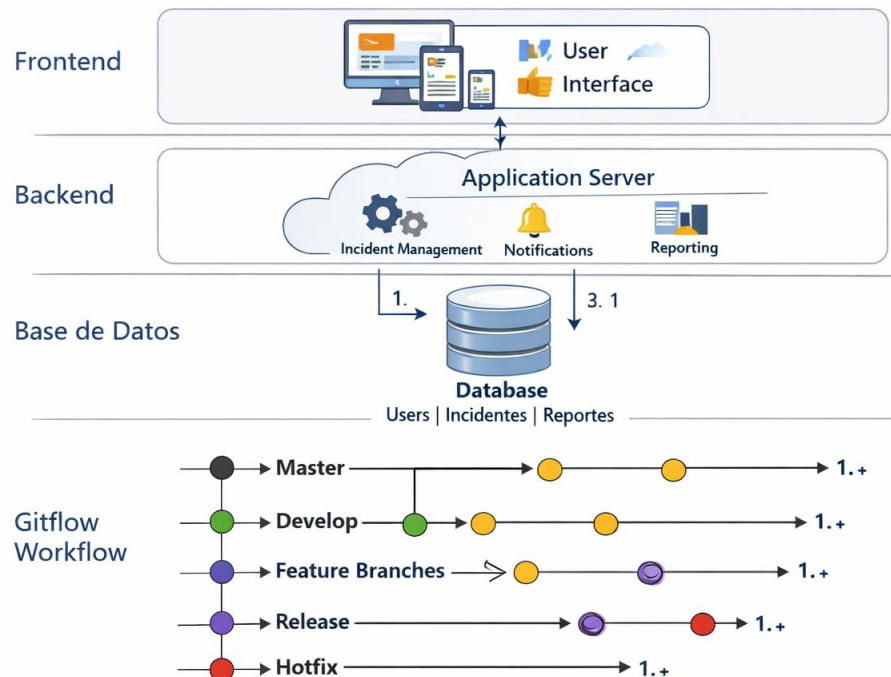
### Capa de integración

- Comunicación entre servicios mediante APIs REST o mensajería asíncrona (RabbitMQ/Kafka).
- Permite la trazabilidad y escalabilidad del sistema.

### Flujo de versiones Gitflow integrado

| Rama     | Propósito                                                          | Flujo de trabajo                                       |
|----------|--------------------------------------------------------------------|--------------------------------------------------------|
| master   | Contiene versiones estables y desplegadas en producción.           | Solo se actualiza desde develop tras pruebas exitosas. |
| develop  | Rama de integración donde se reúnen nuevas funcionalidades.        | Recibe merges desde ramas feature/.                    |
| feature/ | Ramas temporales para desarrollar nuevas funciones o correcciones. | Se crean desde develop y se eliminan tras el merge.    |
| release  | Preparación de versiones antes de pasar a producción.              | Se prueba y documenta antes de fusionar con master.    |
| hotfix   | Corrección urgente de errores en producción.                       | Se crea desde master y luego se fusiona con develop.   |

### Arquitectura del Sistema con Flujo Gitflow



**Frontend:** Es la interfaz gráfica en la que los usuarios finales y soporte técnico interactúan con el sistema para crear, consultar, modificar y seguir los incidentes reportados.

**Backend:** Es el servidor de aplicación que contiene la lógica del sistema, integrando los módulos de incidentes, autenticación, notificaciones y reportes.

**Base de datos:** Es el encargado de contener toda la información del sistema; usuarios, tickets de incidentes, estados, historial de seguimiento y reportes generados.

**Flujo Gitflow:** El flujo se basa en las siguientes ramas principales master, develop, feature, release y hotfix, conectadas mediante flechas que representan el ciclo de desarrollo, integración, pruebas, corrección y despliegue del software de forma organizada y en equipo.

## 4. Aplicación de patrones de diseño

### 4.1 Singleton

Se aplica en el núcleo del motor del sistema ticketing y en el módulo el cual controla la conexión con la base de datos del sistema.

Asegura la existencia de una sola instancia del controlador primario para proveer una administración centralizada, eficiente y segura de los incidentes y una administración segura y consistente de las conexiones y recursos de datos.

### Ejemplo conceptual:

```
clase pública GestorIncidentes {
    GestorIncidentes estático privado instancia;
    privado GestorIncidentes() {}
    public static GestorIncidentes getInstancia() {
        if (instancia == null) instancia = new
GestorIncidentes();
        instancia de devolución ;
    }
}
```

Preserva los recursos del sistema y la coherencia durante el acceso y manipulación de los datos, mejora el rendimiento del sistema y nos permite administrar la información de forma centralizada y consistente.

## 4.2 Observer

Se aplica en el sistema de notificaciones automáticas para informar a los técnicos de soporte sobre la creación, actualización o cambio de estado de los incidentes registrados. Implementar un mecanismo reactivo y desacoplado que permita a los técnicos suscritos recibir alertas en tiempo real cada vez que se genere o modifique un ticket, mejorando la comunicación y la rapidez en la atención de incidentes.

- **Ejemplo conceptual:**

```
Interfaz Observador {
    void actualizar(Incidente incidente);
}

clase Tecnico implementa Observador {
    public void actualizar(Incidente incidente) {
        Sistema . afuera . println( "Nuevo
incidente asignado: " + incidente .
getDescripcion());
    }
}
```

Separa el sistema de notificaciones del módulo de gestión de incidentes, facilitando la extensibilidad, el mantenimiento y añadido de nuevos canales de comunicación sin influir en el funcionamiento principal del sistema.

### 4.3 Factory Method

Aplicado en la generación de incidentes de distinto tipo en base al área de soporte, Red, Hardware y Software, se puede disponer de objetos especializados de manera ordenada y homogénea.

El propósito es centralizar y abstraer la fabricación de objetos para no revelar la lógica de cómo son creados y para mejorar la escalabilidad y mantenimiento de la aplicación.

#### Ejemplo conceptual:

```
class abstracta CreadorIncidente {  
    resumen Incidente crearIncidente();  
}  
  
class CreadorRed extiende CreadorIncidente {  
    Incidente crearIncidente() { return new  
    IncidenteRed(); }  
}
```

Facilita la incorporación de nuevos tipos de incidentes sin necesidad de modificar el código ya existente, promoviendo la escalabilidad, reutilización y mantenibilidad del sistema conforme evolucionen los requerimientos.

### 4.4 MVC (Model-View-Controller)

Se encuentra implementado como patrón arquitectónico para organizar de manera eficiente la lógica de negocio, la gestión de datos y la interacción con la interfaz del sistema.

Separa las responsabilidades entre la presentación, el control de procesos y la administración de datos, favoreciendo una arquitectura modular y de fácil mantenimiento.

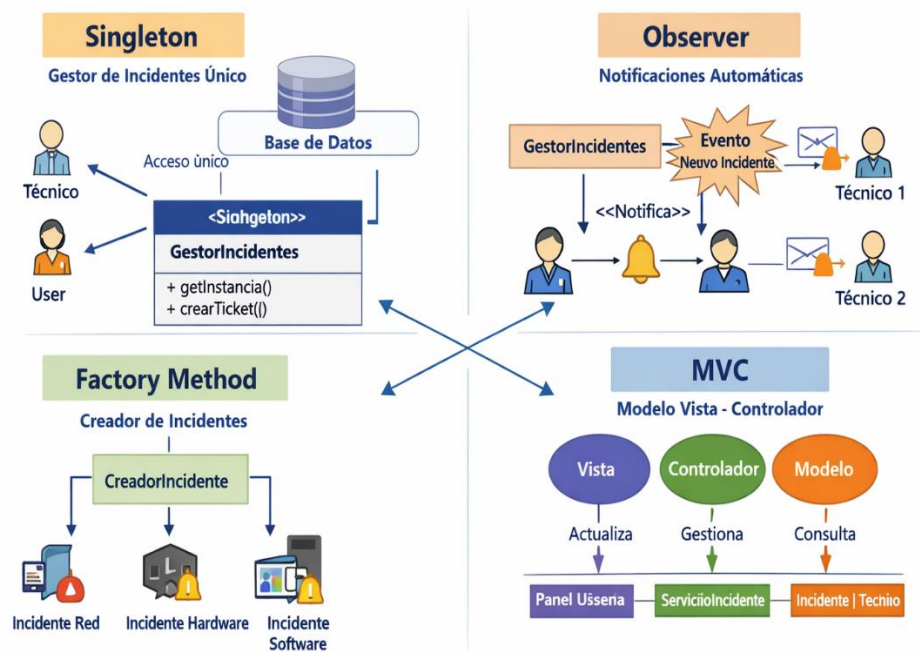
#### Estructura:

- **Model:** conformado por las clases Usuario, Tecnico e Incidente, encargadas de representar y gestionar los datos y reglas de negocio del sistema.
- **View:** interfaz web o panel de control mediante el cual usuarios y técnicos interactúan con el sistema para registrar, consultar y administrar incidentes.

- **Controller:** clase ServicioIncidente, responsable de coordinar la comunicación entre la vista y el modelo, procesando solicitudes y ejecutando las operaciones correspondientes.

Incrementa la mantenibilidad, escalabilidad y capacidad de prueba del sistema al separar claramente las responsabilidades de cada componente, permitiendo realizar modificaciones, actualizaciones o ampliaciones de manera independiente sin comprometer el funcionamiento general de la aplicación.

### Patrones de Diseño en Gestión de Incidentes TI



## 5. Documentación del diseño

### 5.1 Propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.

### 5.1.1 Singleton

- Permitir que el motor del gestor de tickets y el driver que conecta con la base de datos ejecuten una única instancia, controlando así los recursos del sistema de forma centralizada.
- Puede evitar conflictos y posibles inconsistencias al tener a varios usuarios o técnicos conectados en tiempo real al sistema de Gestión de incidencia.
- La clase GestorIncidentes tiene centralizada la alta, baja y seguimiento de tickets para que todos los técnicos trabajen sobre una única fuente de datos que está actualizada y sincronizada al instante.
- Proporciona control unificado del flujo de incidentes, administración más eficiente de recursos e información y consistencia global.

### 5.1.2 Observer

Diseñar un sistema de notificación automática, reactiva y desacoplada para que los técnicos estén al día con los cambios en los incidentes registrados.

No se comprueba manualmente el sistema para ver si hay nuevos incidentes porque las alertas se crean y envían automáticamente en tiempo real.

Cuando un ticket es creado, modificado o despachado, la clase GestorIncidentes envía una notificación a todos los técnicos suscritos siguiendo el patrón Observer.

Este facilita la comunicación interna, acelera la atención de incidentes críticos y otorga una respuesta más rápida y efectiva al fallo o requerimiento de soporte.

### 5.1.3 Factory Method

Posibilitar que se creen distintos tipos de incidentes en función del área de soporte, ya sea Red, Hardware o Software sin afectar la estructura general del sistema.

Se elimina la dependencia hacia clases concretas, haciendo que la introducción de nuevos tipos de incidentes sea más sencilla y que el impacto de los cambios en el código existente también sea menor.



La clase CreadorIncidente produce objetos específicos en función de la categoría escogida por el usuario. atributos y comportamientos específicos de cada tipo de hijo de Incidente.

Proporciona flexibilidad, capacidad para crecer y para ampliarse en el sistema para nuevas áreas de soporte, nuevas tecnologías o nuevos tipos de fallos, sin que se vea comprometida la mantenibilidad de aplicación.

#### **5.1.4 MVC (Model-View-Controller)**

Separar claramente y de manera organizada entre la capa de lógica de negocios, capa de interfaz de usuario, y capa de control de flujo de aplicación, y fomentar la construcción de una arquitectura con carga modular, estructurada y fácil de mantenimiento.

Previene que modificaciones a la interfaz gráfica tengan impacto directo en la lógica de negocio o en la gestión de la base de datos, disminuyendo el acoplamiento entre módulos y aumentando la independencia funcional del sistema.

Está compuesto por las entidades Usuario, Técnico e Incidente que representan los datos y contienen las reglas de negocio del sistema. o Vista: panel web del sistema Help Desk para que los usuarios y técnicos puedan registrar, consultar y administrar los incidentes.

La clase ServicioIncidente que funciona como controlador en lo que respecta a la lógica de presentación del modelo vista controlador, manejando solicitudes y realizando operaciones.

Mejora la mantenibilidad, escalabilidad y reutilización del sistema, así como también permite las pruebas, actualizaciones y potencial expansión con mínimo o nulo impacto en el funcionamiento global de la aplicación.

## **5.2 Justificación del uso de Gitflow**

Se implementa el modelo Gitflow para tener un control ordenado y estructurado del desarrollo del sistema, normalizando un buen control de versiones, trazabilidad de



cambios, colaboratividad entre el equipo y estabilidad en cada etapa de desarrollo y despliegue de la aplicación.

### 5.2.1 Ventajas principales

Organización del desarrollo: se estructura el trabajo en ramas específicas, dichas ramas usualmente son feature, develop y master, con esto se logra administrar nuevas funcionalidades, pruebas y versiones de una forma ordenada y se disminuye la probabilidad de conflicto entre cambios.

Colaboración entre varios desarrolladores que puede trabajar en paralelo en múltiples funcionalidades o arreglos sin afectar la línea base del proyecto.

Trazabilidad a nivel versiones: cada commit, merge y update queda registrado en el historial del repo permitiendo un seguimiento detallado, realizar auditorías, etc.

Estabilidad en producción: la rama master sólo contiene releases validadas, testeadas y listas para ser desplegadas, lo que provee mayor confiabilidad y seguridad en producción.

### 5.2.2 Estructura de ramas en tu repositorio

| Rama     | Propósito                                                                      | Ejemplo de uso                                                  |
|----------|--------------------------------------------------------------------------------|-----------------------------------------------------------------|
| master   | Versión estable del sistema Help Desk.                                         | commit inicial: subida de la primera versión de la presentación |
| develop  | Rama de integración donde se reúnen nuevas funcionalidades.                    | merge github                                                    |
| feature/ | Ramas temporales para avances específicos (ejemplo: módulo de notificaciones). | feature/notificaciones-automaticas                              |

### 5.2.3 Enlace al repositorio Publico

URL del depositario creado en GitHub del proyecto desarrollado.

<https://github.com/marcelmismael/DesarrollossoftwareMarcellmael.git>



URL de la estructura de los grafitos

```
https://lucid.app/lucidchart/0b096a67-304f-49fd-92de-  
d19a7579c254/edit?viewport_loc=-192%2C  
84%2C1436%2C681%2Cy5wXHDsl54Jv&invitationId=inv_619a939d-  
acfa-4f5e-a706-1a2ce8904766
```

#### 5.2.4 Evidencia del flujo de trabajo

Las siguientes capturas muestran el historial de commits y ramas del proyecto:

##### 5.2.4.1 Creación del repositorio y primer commit

git init y commit inicial:

```
MINGW64:/c:/Users/User/Desktop/PROYECTO2026
User@DESKTOP-AT91HB2 MINGW64 ~ (master)
$ cd ~/Desktop/PROYECTO2026

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ ls

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ ls
Actividad_1_Desarrollo_Sistemas_Informaticos_Marcel_Sinche.pdf

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ git add .

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ git commit -m "Se agrega el archivo PDF"
[master (root-commit) a731144] Se agrega el archivo PDF
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Actividad_1_Desarrollo_Sistemas_Informaticos_Marcel_Sinche.p
df

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$
```

Rama activa: **master**.

##### 5.2.4.2 Configuración de ramas Gitflow

git checkout -b develop

git push -u origin develop

|                                                       |        |     |        |
|-------------------------------------------------------|--------|-----|--------|
| git                                                   | remote | add | origin |
| https://github.com/marcelmismael/DesarrolloSoftwareMa |        |     |        |
| rcel-.git                                             |        |     |        |



```
MINGW64:/c/Users/User/Desktop/PROYECTO2026

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ git checkout -b develop
Switched to a new branch 'develop'

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (develop)
$ git remote add origin https://github.com/marcelmismael/DesarrolloSoftwareMarcel-1-.git

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (develop)
$ git push -u origin develop
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.80 MiB | 875.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/marcelmismael/DesarrolloSoftwareMarcel-1-/pull/new/develop
remote:
To https://github.com/marcelmismael/DesarrolloSoftwareMarcel-1-.git
 * [new branch]      develop -> develop
```

Se crean ramas feature/ para avances específicos.

#### 5.2.4.3 Sincronización con GitHub

```
git merge github
```

```
git push -u origin main
```

```
MINGW64/c/Users/User/Desktop/PROYECTO2026
$ git checkout -b develop
Switched to a new branch 'develop'

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (develop)
$ git remote add origin https://github.com/marcelismael/DesarrolloSoftwareMarcel-.git

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (develop)
$ git push -u origin develop
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.80 MiB | 875.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/marcelismael/DesarrolloSoftwareMarcel-/pull/new
remote: /develop
remote:
To https://github.com/marcelismael/DesarrolloSoftwareMarcel-.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (develop)
$ git branch -r
  origin/develop

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (develop)
$ git checkout master
Switched to branch 'master'

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ git push -u origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/marcelismael/DesarrolloSoftwareMarcel-/pull/new
remote: /master
remote:
To https://github.com/marcelismael/DesarrolloSoftwareMarcel-.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$ git branch -r
  origin/develop
  origin/master

User@DESKTOP-AT91HB2 MINGW64 ~/Desktop/PROYECTO2026 (master)
$
```

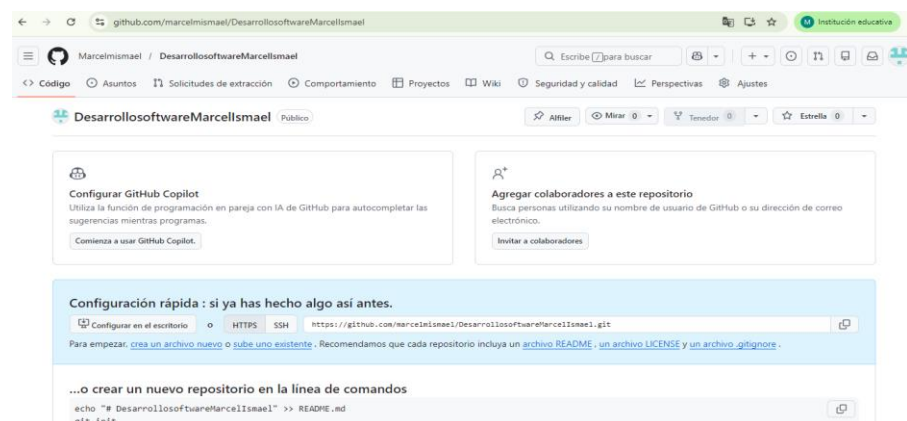
Se evidencia el flujo de integración entre local y remoto.

#### 5.2.4.4 Repositorio público visible

Archivos:

Actividad\_1\_Desarrollo\_Sistemas\_Informaticos\_Marcel\_Sinche.

pdf, Descripción: *DesarrolloSoftwareMarcelIsmael*



## 6. Conclusión

El proyecto de incidentes de gestión de IT todo consolida un modelo robusto que cubre usuarios, técnicos y procesos en un flujo organizado. El usar UML para modelar el diseño ayuda a mostrar claramente las relaciones y responsabilidades de cada clase. La mantenibilidad se garantiza y facilita la incorporación de nuevas funcionalidades gracias a la modularidad. Se dispone de traceability completa desde el reporte hasta la solución del incidente. Se proporciona un fundamento sólido para la aplicación a los entornos educativos y corporativos.

La arquitectura sugerida permite una eficiente gestión integral de incidentes mediante la separación de capas y el control de estados. El repositorio centraliza la persistencia mientras que el notificador garantiza una eficiente comunicación. La clase report ofrece valor estratégico al permitir análisis y auditoría del sistema. El diseño satisface principios de ingeniería de software, como la responsabilidad única y la escalabilidad. El proyecto en su conjunto es una solución robusta y que puede ser aplicada a situaciones reales.



Marcel Ismel Sinche Yumisaca  
0603747445